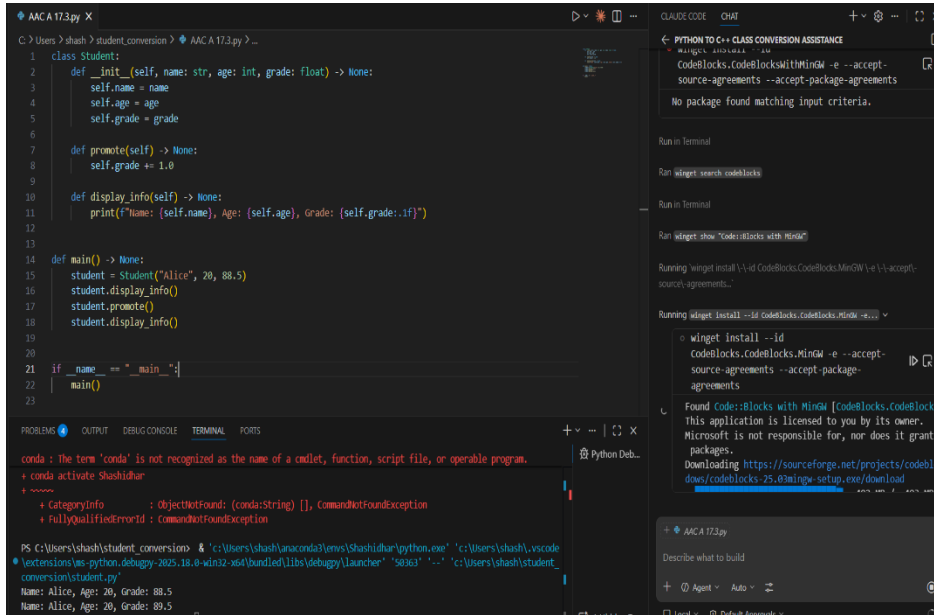


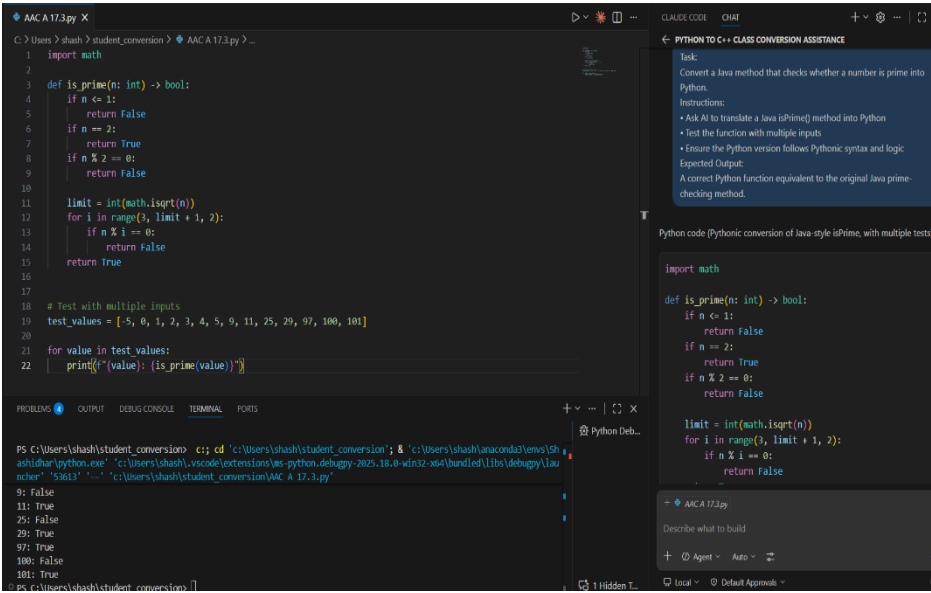
Name: B. Nithin H.No: 2303A51803 Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M. Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week10 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 19.3 (Present assignment number) / 24 (Total number of assignments)			

Q.No.	Question	Expected Time to complete
1	Lab 19 – Code Translation: Converting Between Programming Languages Lab Objectives: Understand how AI tools can assist in translating code between	Week10 – Wednesday

	different programming languages. - Learn to verify correctness and functionality after translation. - Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). - Practice debugging and optimizing AI-translated code.	
	Task 1 – Python to C++ Conversion Task: Translate a simple Python class into C++ using AI assistance. Instructions: - Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code - Ensure proper handling of: - Constructors - Data types - Access specifiers - Compile and run both versions to verify output consistency Expected Output: An equivalent working C++ class translated from Python. 	
	Task 2 – Java to Python Function Conversion Task: Convert a Java method that checks whether a number is prime into Python. Instructions: - Ask AI to translate a Java isPrime() method into Python - Test the function with multiple inputs - Ensure the Python version follows Pythonic syntax and logic	

Expected Output:
A correct Python function equivalent to the original Java prime-checking method.



Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

```
AAC A 17.3.py X
C:\Users\shash> student_conversion > AAC A 17.3.py > ...
1 def bubble_sort(nums):
2     arr = nums[:]
3     n = len(arr)
4     for i in range(n):
5         swapped = False
6         for j in range(0, n - i - 1):
7             if arr[j] > arr[j + 1]:
8                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
9                 swapped = True
10        if not swapped:
11            break
12        return arr
13
14
15 def main():
16     test_lists = [
17         [64, 34, 25, 12, 22, 11, 90],
18         [5, 1, 4, 2, 8],
19         [3, 3, 2, 1, 5, 0],
20         [],
21         [10],
22     ]
23
24     for lst in test_lists:
25         print("Input :", lst)
26         print("Sorted:", bubble_sort(lst))
27         print()
28
29 if __name__ == "__main__":
30     main()
31
32
```

```
def bubble_sort(nums):
    arr = nums[:]
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
        return arr

def main():
    test_lists = [
        [64, 34, 25, 12, 22, 11, 90],
        [5, 1, 4, 2, 8],
        [3, 3, 2, 1, 5, 0],
        [],
        [10],
    ]

    for lst in test_lists:
        print("Input :", lst)
        print("Sorted:", bubble_sort(lst))
        print()

AAC A 17.3.py
Describe what to build
Agent - Done
```

```
PS C:\Users\shash\student_conversion> c:\cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56797' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Input : []
Sorted: []

Input : [10]
Sorted: [10]
```

```
AAC A 17.3.py X
C:\Users\shash> student_conversion > AAC A 17.3.py > ...
15 def main():
16     test_lists = [
17         [64, 34, 25, 12, 22, 11, 90],
18         [5, 1, 4, 2, 8],
19         [3, 3, 2, 1, 5, 0],
20         [],
21         [10],
22     ]
23
24     for lst in test_lists:
25         print("Input :", lst)
26         print("Sorted:", bubble_sort(lst))
27         print()
28
29 if __name__ == "__main__":
30     main()
31
32
```

```
PS C:\Users\shash\student_conversion> c:\cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56797' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Input : []
Sorted: []

Input : [10]
Sorted: [10]
```

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

The screenshot shows a VS Code editor with a Python file named 'AAC A 17.3.py'. The code defines a DataFrame 'employees' with columns 'name', 'salary', and 'department'. It then filters the DataFrame to show only employees with a salary greater than 50,000. The output of the script is displayed in the terminal window at the bottom, showing the filtered data as a table with 4 rows and 2 columns: name and salary.

```
1 import pandas as pd
2
3 employees = pd.DataFrame({
4     'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Evan'],
5     'salary': [48000, 62000, 51000, 45000, 75000],
6     'department': ['HR', 'Engineering', 'Sales', 'Finance', 'Engineering'],
7 })
8
9
10
11 result = employees.loc[employees['salary'] > 50000, ['name', 'salary']]
12
13 print(result)
```

The terminal output shows the following data:

name	salary
Bob	62000
Charlie	51000
Evan	75000

On the right side of the editor, there is an AI assistant window titled 'PYTHON TO C++ CLASS CONVERSION ASSISTANCE'. It contains a task description: 'Task 4 - SQL to Pandas Query' and instructions to translate a SQL query into a Pandas DataFrame operation in Python. The assistant also provides a sample DataFrame and the expected output, which matches the terminal output shown in the screenshot.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

ACA 17.3.py X

```
C:\Users\shash> student_conversion > ACA 17.3.py >
1 def python_binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = left + (right - left) // 2
5         if arr[mid] == target:
6             return mid
7         if arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1
12
13
14 def c_style_binary_search(arr, size, target):
15     left = 0
16     right = size - 1
17     while left <= right:
18         mid = left + (right - left) // 2
19         if arr[mid] == target:
20             return mid
21         elif arr[mid] < target:
22             left = mid + 1
23         else:
24             right = mid - 1
25    return -1
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda\envs\shash\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bin\debugpy\launcher' '4000' '-c' 'c:\Users\shash\student_conversion\ACA 17.3.py'
Search Tests
Input: [64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True
Input: [5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True
Input: [3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True
Input: [], java_like=[], js_like=[], match=True
Input: [10], java_like=[10], js_like=[10], match=True
PS C:\Users\shash\student_conversion>
```

CLAUDE CODE CHAT

PYTHON TO C++ CLASS CONVERSION ASSISTANCE

Task 5 - Real-Time Application: Algorithm Translation Across Languages
Scenario:
A company maintains algorithms written in different programming languages.
Examples:
• Converting a Python searching algorithm into C
• Translating a Java sorting program into JavaScript
Instructions:
• Use AI to translate a selected algorithm between two programming languages
• Exercise and test both versions
• Document translation challenges such as:
o Syntax differences
o Library support
o Memory management
Expected Output:
Equivalent, tested code implementations in two different programming languages
give code in python without comments

def python_binary_search(arr, target):
 left, right = 0, len(arr) - 1
 while left <= right:
 mid = left + (right - left) // 2
 if arr[mid] == target:
 return mid
 elif arr[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
 return -1

def c_style_binary_search(arr, size, target):
 left = 0
 right = size - 1
 while left <= right:
 mid = left + (right - left) // 2
 if arr[mid] == target:
 return mid
 elif arr[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
 return -1

+ ACA 17.3.py
Describe what to build
+ Agent Auto
Local Default Approvals

ACA 17.3.py X

```
C:\Users\shash> student_conversion > ACA 17.3.py >
14 def c_style_binary_search(arr, size, target):
15     left = 0
16     right = size - 1
17     while left <= right:
18         mid = left + (right - left) // 2
19         if arr[mid] == target:
20             return mid
21         elif arr[mid] < target:
22             left = mid + 1
23         else:
24             right = mid - 1
25    return -1
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda\envs\shash\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bin\debugpy\launcher' '4000' '-c' 'c:\Users\shash\student_conversion\ACA 17.3.py'
Search Tests
Input: [64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True
Input: [5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True
Input: [3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True
Input: [], java_like=[], js_like=[], match=True
Input: [10], java_like=[10], js_like=[10], match=True
PS C:\Users\shash\student_conversion>
```

CLAUDE CODE CHAT

PYTHON TO C++ CLASS CONVERSION ASSISTANCE

def python_binary_search(arr, target):
 left, right = 0, len(arr) - 1
 while left <= right:
 mid = left + (right - left) // 2
 if arr[mid] == target:
 return mid
 elif arr[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
 return -1

def c_style_binary_search(arr, size, target):
 left = 0
 right = size - 1
 while left <= right:
 mid = left + (right - left) // 2
 if arr[mid] == target:
 return mid
 elif arr[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
 return -1

+ ACA 17.3.py
Describe what to build
+ Agent Auto
Local Default Approvals

```
AAC A 17.3.py X
C: > Users > shash > student_conversion > AAC A 17.3.py > ...

40 def javascript_bubble_sort(arr):
41     a = arr[:]
42     n = len(a)
43     for i in range(n):
44         swapped = False
45         for j in range(0, n - i - 1):
46             if a[j] > a[j + 1]:
47                 a[j], a[j + 1] = a[j + 1], a[j]
48                 swapped = True
49         if not swapped:
50             break
51     return a
52
53
54 def run_search_tests():
55     data = [3, 7, 12, 19, 24, 31, 45, 58]
56     targets = [3, 24, 58, 9]
57     print("Search Tests")
58     for t in targets:
59         p = python_binary_search(data, t)
60         c = c_style_binary_search(data, len(data), t)
61         print(f"target={t}, python={p}, c_style={c}, match={p == c}")
62
63
64 def run_sort_tests():
65     test_cases = [
66         [64, 34, 25, 12, 22, 11, 90],
67         [5, 1, 4, 2, 8],
68         [3, 3, 2, 1, 5, 0],
69         [],
70         [10],
71     ]
72     print("\nSort Tests")
73     for case in test_cases:
74         j = java_bubble_sort(case)
75         js = javascript_bubble_sort(case)
76         print(f"input={case}, java_like={j}, js_like={js}, match={j == js}")
77
78
79 if __name__ == "__main__":
80     run_search_tests()
81     run_sort_tests()
82
```

PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Search Tests

input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True

input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True

input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True

input=[], java_like=[], js_like=[], match=True

input=[10], java_like=[10], js_like=[10], match=True

PS C:\Users\shash\student_conversion>

```
AAC A 17.3.py X
C: > Users > shash > student_conversion > AAC A 17.3.py > ...

63
64 def run_sort_tests():
65     test_cases = [
66         [64, 34, 25, 12, 22, 11, 90],
67         [5, 1, 4, 2, 8],
68         [3, 3, 2, 1, 5, 0],
69         [],
70         [10],
71     ]
72     print("\nSort Tests")
73     for case in test_cases:
74         j = java_bubble_sort(case)
75         js = javascript_bubble_sort(case)
76         print(f"input={case}, java_like={j}, js_like={js}, match={j == js}")
77
78
79 if __name__ == "__main__":
80     run_search_tests()
81     run_sort_tests()
82
```

PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Search Tests

input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True

input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True

input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True

input=[], java_like=[], js_like=[], match=True

input=[10], java_like=[10], js_like=[10], match=True

PS C:\Users\shash\student_conversion>

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.